By Prelin Govender

Student Lifestyle and Performance Data Analysis

Dataset Description : This is a dataset that contains the record of 2000 university students, it was obtained via open datasets on Kaggle.com. The data focuses on the daily activities of the students, time allocations per activies and the resulting academic performance.

Dataset Variables : Student ID, Study hours, Extracurricular hours, Sleep hours, Physical Activity, GPA, Stress level

Goals :

- To explore and familiarise myself with data science methodlogy (Data acquistion, preparation, analysis, modeling & evaluation).
- Learn the basic to advanced application of data science tools (Pandas, Numpy, Scikit-learn, Matplotlib, Tensorflow).
- With regard to the data, identifying the relationship between daily activities and academic perfomance.
- Determine the magnitude of individual activites on academic performance or student's stress levels.

Before the process beings, we can initially import the core data handling and visualization libraries.

In [1]:
```
#Python libraries that are key to the task are imported, the "as" term allows us to redefine the library function
#call to a shorter form.
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

print("Core libraries imported and configured successfully!")
```

```
Core libraries imported and configured successfully!
```

STEP 1 : Importing the dataset and confirming if it is loaded properly.

In [2]:
```
#The csv file is read using a pandas function.
df = pd.read_csv('student_lifestyle_dataset.csv')
```

In [3]:
```
#The head and description of the dataset can be called to view if the dataset was imported correctly.
print("DataFrame Head:")
print(df.head())
```

```
print("\nDataFrame Info:")
df.info()
```

```
DataFrame Head:
   Student_ID  Study_Hours_Per_Day  Extracurricular_Hours_Per_Day  \
0           1                  6.9                            3.8
1           2                  5.3                            3.5
2           3                  5.1                            3.9
3           4                  6.5                            2.1
4           5                  8.1                            0.6

   Sleep_Hours_Per_Day  Social_Hours_Per_Day  Physical_Activity_Hours_Per_Day  \
0                  8.7                   2.8                              1.8
1                  8.0                   4.2                              3.0
2                  9.2                   1.2                              4.6
3                  7.2                   1.7                              6.5
4                  6.5                   2.2                              6.6

    GPA Stress_Level
0  2.99     Moderate
1  2.75          Low
2  2.67          Low
3  2.88     Moderate
4  3.51         High

DataFrame Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2000 entries, 0 to 1999
Data columns (total 8 columns):
 #   Column                          Non-Null Count  Dtype
---  ------                          --------------  -----
 0   Student_ID                      2000 non-null   int64
 1   Study_Hours_Per_Day             2000 non-null   float64
 2   Extracurricular_Hours_Per_Day   2000 non-null   float64
 3   Sleep_Hours_Per_Day             2000 non-null   float64
 4   Social_Hours_Per_Day            2000 non-null   float64
 5   Physical_Activity_Hours_Per_Day  2000 non-null   float64
 6   GPA                             2000 non-null   float64
 7   Stress_Level                    2000 non-null   object
dtypes: float64(6), int64(1), object(1)
memory usage: 125.1+ KB
```

In [4]:
```
#It is also possible to view the table with all columns, however with a custom number of rows so that not all
#entries are printed.
print(df.head(10).to_markdown(index=False, numalign="left", stralign="left"))
```

Student_ID	Study_Hours_Per_Day	Extracurricular_Hours_Per_Day	Sleep_Hours_Per_Day	Social_Hours_Per_Day	Physical_Activity_Hours_Per_Day	GPA	Stress_Level
1	6.9	3.8	8.7	2.8	1.8	2.99	Moderate
2	5.3	3.5	8	4.2	3	2.75	Low
3	5.1	3.9	9.2	1.2	4.6	2.67	Low
4	6.5	2.1	7.2	1.7	6.5	2.88	Moderate
5	8.1	0.6	6.5	2.2	6.6	3.51	High
6	6	2.1	8	0.3	7.6	2.85	Moderate
7	8	0.7	5.3	5.7	4.3	3.08	High
8	8.4	1.8	5.6	3	5.2	3.2	High
9	5.2	3.6	6.3	4	4.9	2.82	Low
10	7.7	0.7	9.8	4.5	1.3	2.76	Moderate

In [5]:
```
#Another method is retrieving only certain columns as desired alongsided the custom number of rows
print(df[['Student_ID', 'GPA', 'Stress_Level']].tail(15).to_markdown(index=False, numalign="left", stralign="left"))
```

Student_ID	GPA	Stress_Level
1986	2.93	Moderate
1987	2.9	High
1988	3.86	High
1989	3.28	Moderate
1990	2.85	Moderate
1991	3.26	High
1992	3.21	High
1993	3.04	Moderate
1994	2.85	Moderate
1995	3.08	Moderate
1996	3.32	Moderate
1997	2.65	Moderate
1998	3.14	Moderate
1999	3.04	High
2000	3.58	High

STEP 2 : Determining if the dataset requires cleaning before data exploration begins.

In [6]:
```
#Checking if there are any empty cells within the dataset table.
print("Missing Values Check:")
print(df.isnull().sum())
```

```
Missing Values Check:
Student_ID                        0
Study_Hours_Per_Day               0
Extracurricular_Hours_Per_Day     0
Sleep_Hours_Per_Day               0
Social_Hours_Per_Day              0
Physical_Activity_Hours_Per_Day   0
GPA                               0
Stress_Level                      0
dtype: int64
```

In [7]:
```
#Checking if the datatypes within each column are consistent with each other.
print("\nData Types Check:")
df.info()
```

```
Data Types Check:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2000 entries, 0 to 1999
Data columns (total 8 columns):
 #   Column                          Non-Null Count  Dtype  
---  ------                          --------------  -----  
 0   Student_ID                      2000 non-null   int64  
 1   Study_Hours_Per_Day             2000 non-null   float64
 2   Extracurricular_Hours_Per_Day   2000 non-null   float64
 3   Sleep_Hours_Per_Day             2000 non-null   float64
 4   Social_Hours_Per_Day            2000 non-null   float64
 5   Physical_Activity_Hours_Per_Day 2000 non-null   float64
 6   GPA                             2000 non-null   float64
 7   Stress_Level                    2000 non-null   object 
dtypes: float64(6), int64(1), object(1)
memory usage: 125.1+ KB
```

In [8]:
```
#Checking if there are any duplicate entries in the dataset table.
duplicate_count = df.duplicated().sum()
print(f"\nDuplicate Rows Check: {duplicate_count} duplicate rows found.")
```

```
Duplicate Rows Check: 0 duplicate rows found.
```

Dataset does not possess issues that could be problematic during data exploration ! However it is important to note that that the data cleaning step will usually take more time and effort when working with more larger and advanced datasets.

STEP 3 : Beginning data exploration to determine the various relationships between variables in the dataset.

Let's begin by calculating the descriptive stats of the dataset.

In [9]:
```
#First, we need to isolate the columns with numeric data while also removing the student ID column.
numeric_cols = df.select_dtypes(include=['float64', 'int64']).columns.drop('Student_ID')

#The remainder of the table is transposed to make viewing easier.
descriptive_stats = df[numeric_cols].describe().T

#The dataset table is summarised into a descriptive stats table.
print("Descriptive Statistics for Numeric Columns:")
print(descriptive_stats.to_markdown(numalign="left", stralign="left"))
```

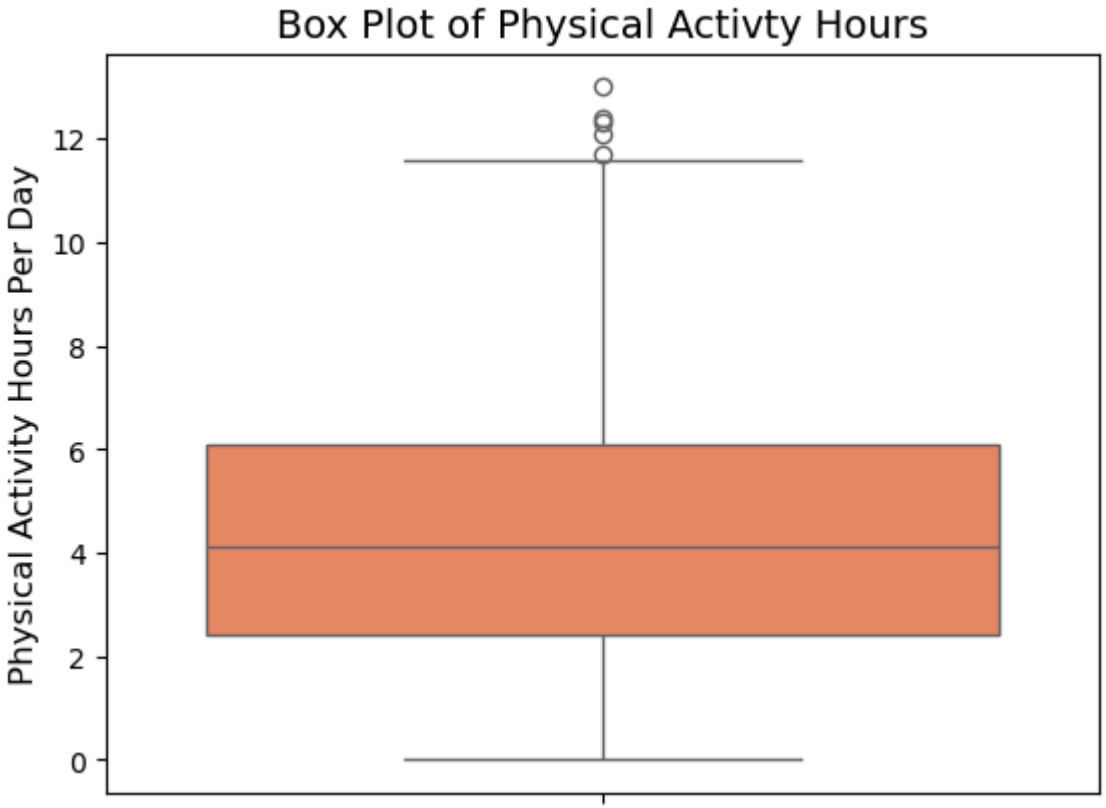
```
Descriptive Statistics for Numeric Columns:
|                                | count   | mean    | std       | min   | 25%   | 50%   | 75%   | max   |
|:-------------------------------|:--------|:--------|:----------|:------|:------|:------|:------|:------|
| Study_Hours_Per_Day            | 2000    | 7.4758  | 1.42389   | 5     | 6.3   | 7.4   | 8.7   | 10    |
| Extracurricular_Hours_Per_Day  | 2000    | 1.9901  | 1.15585   | 0     | 1     | 2     | 3     | 4     |
| Sleep_Hours_Per_Day            | 2000    | 7.50125 | 1.46095   | 5     | 6.2   | 7.5   | 8.8   | 10    |
| Social_Hours_Per_Day           | 2000    | 2.70455 | 1.68851   | 0     | 1.2   | 2.6   | 4.1   | 6     |
| Physical_Activity_Hours_Per_Day | 2000    | 4.3283  | 2.51411   | 0     | 2.4   | 4.1   | 6.1   | 13    |
| GPA                            | 2000    | 3.11596 | 0.298674  | 2.24  | 2.9   | 3.11  | 3.33  | 4     |
```

Here are some observations from the descriptive stats:

- The average GPA score is slightly high, while the standard deviation is significantly low, this infers that the GPA scores are closely clustered around the mean and displays consistent data.
- The range of the study hours is from 5 to 10 hours, the students utilise a significant portion of their day to their studies. The mean and the 50% (median) are very close in values which singals that the distribution of data is symmetrical and skewed either way.
- The sleep hour data is also very symmetrical, the average sleep hour is 7.5hrs therefore students are generally getting enough rest and that this aspect might not be a heavy contributor to performance variation in the students.
- The standard deviation for physcial activy is higher than others which shows that it has higher variability amongst students. There is also a maximum value of 13 which is more than twice the 75th percentile indicating that this is a potential outlier or error.
- The extracurricular hours and spcial hours have a relatively low mean, which possibly suggests that they have low impact of GPA performance but will need further investigation to be sure.

Now let us investigate the potential outlier entry, we can use the box plot grapgh to confirm the 13 hours of physical activity should be marked as an outlier.

In [10]:
```
#The box plot can be created with custom characterisitcs
sns.boxplot(y=df['Physical_Activity_Hours_Per_Day'], color='coral')
plt.title('Box Plot of Physical Activty Hours', fontsize=14)
plt.ylabel('Physical Activity Hours Per Day', fontsize=12)

plt.savefig('physical_activity_boxplot.png')
plt.show()
```



The box plot of the variable is interpreted as such:

- The box of the figure represents 25th percentile to the 75th percentile, which is the middle 50% of the total data.
- Inside of the box, the line represents the 50th percentile.
- The whiskers of the figure represent the extension of the standard range of the data.
- The tiny circles we see at the top represent the statistical outliers of the dataset, which confirms our theory about the value 13 but also indicates to us that there are several other outliers.

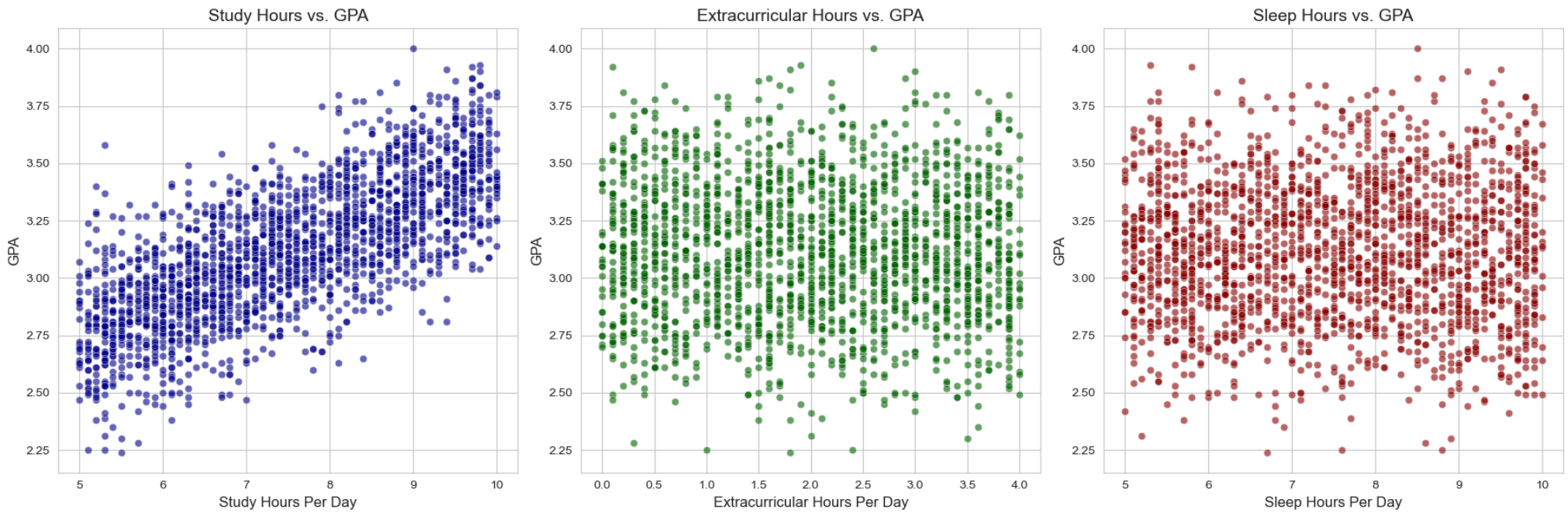
What are the options we have to deal with outliers?

- We can keep them, theorizing that they represent extra behaviours of students (eg. marathon training, sports events) and it is still relevant to the data.
- Delete the outliers through going back into the data cleaning process, which can improve the perfomance of oru data models in the future at the cost of the size of the dataset.
- Transform the column data using a mathematical logarithm to reduce the impact of the outliers without having to remove them.

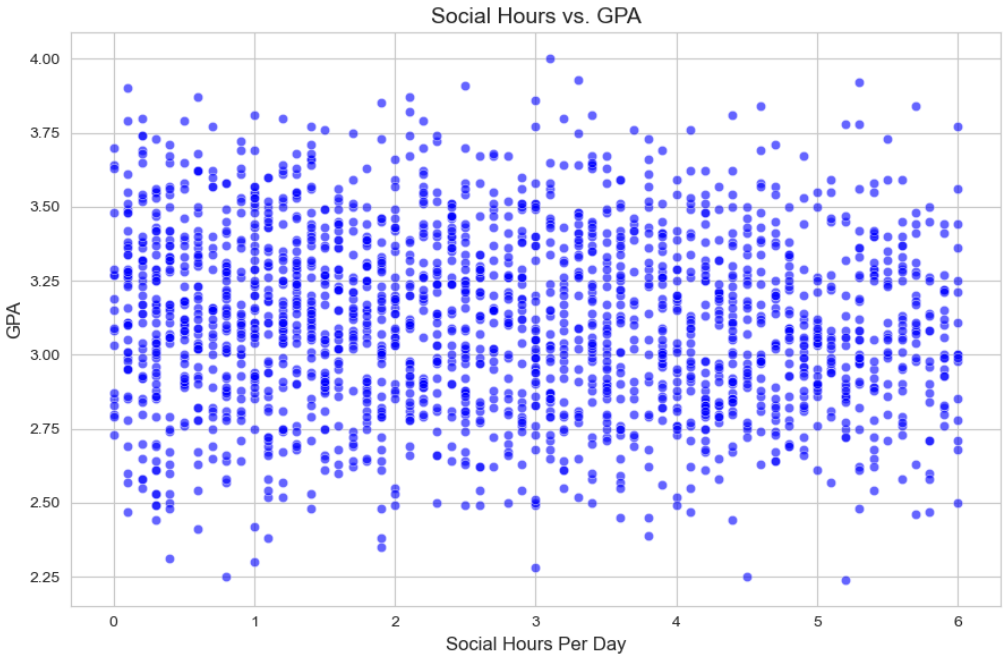
For this example dataset, we shall keep the outliers. We will explore the data cleaing and exploration iterative process in future example datasets, while using the outliers in this set to try and understand their influence on data models.

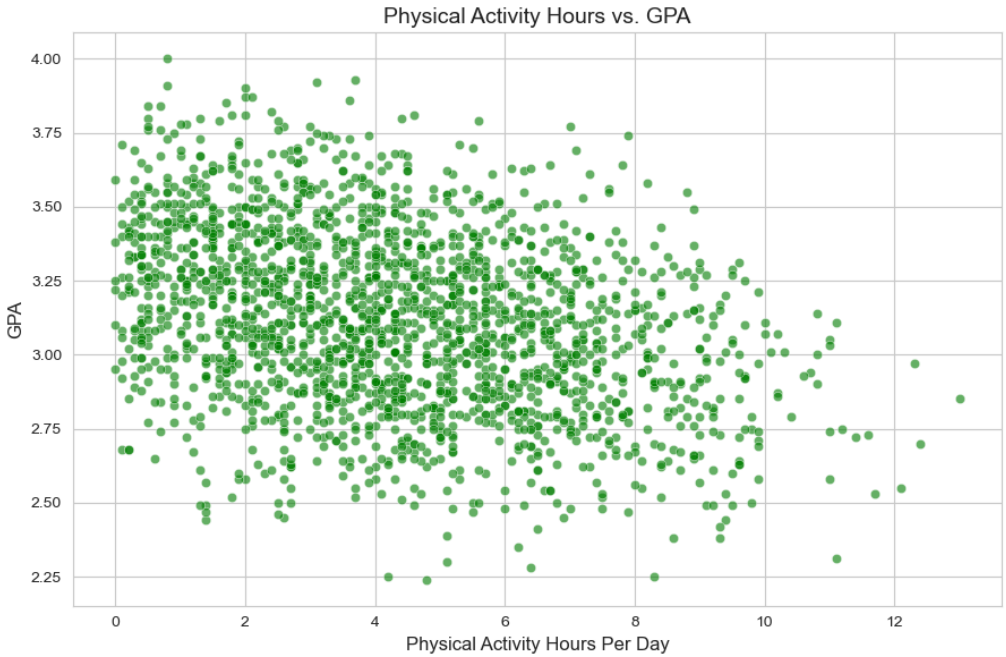
Now that we have investigated and confirmed outliers, let us explore the relationships between the variables. A scatter plot can be used to visually represent the relationship between two continous variables and allow us to detect correlations and patterns between them.

In [11]:
```
#Setting up a professional visualization style.
sns.set_style('whitegrid')

#Seting up a plot with 3 subplots of variables
fig, axes = plt.subplots(1, 3, figsize=(18, 6))

sns.scatterplot(x='Study_Hours_Per_Day', y='GPA', data=df, ax=axes[0], alpha=0.6, color='darkblue')
axes[0].set_title('Study Hours vs. GPA', fontsize=14)
axes[0].set_xlabel('Study Hours Per Day', fontsize=12)
axes[0].set_ylabel('GPA', fontsize=12)

sns.scatterplot(x='Extracurricular_Hours_Per_Day', y='GPA', data=df, ax=axes[1], alpha=0.6, color='darkgreen')
axes[1].set_title('Extracurricular Hours vs. GPA', fontsize=14)
axes[1].set_xlabel('Extracurricular Hours Per Day', fontsize=12)
axes[1].set_ylabel('GPA', fontsize=12)

sns.scatterplot(x='Sleep_Hours_Per_Day', y='GPA', data=df, ax=axes[2], alpha=0.6, color='darkred')
axes[2].set_title('Sleep Hours vs. GPA', fontsize=14)
axes[2].set_xlabel('Sleep Hours Per Day', fontsize=12)
axes[2].set_ylabel('GPA', fontsize=12)

plt.tight_layout()
plt.savefig('key_scatter_plots.png')
plt.show()
```



In [12]:
```
#Seting up a plot with the remaining 2 subplots of variables
fig, axes = plt.subplots(1, 2, figsize=(18, 6))

sns.scatterplot(x='Social_Hours_Per_Day', y='GPA', data=df, ax=axes[0], alpha=0.6, color='blue')
axes[0].set_title('Social Hours vs. GPA', fontsize=14)
axes[0].set_xlabel('Social Hours Per Day', fontsize=12)
axes[0].set_ylabel('GPA', fontsize=12)

sns.scatterplot(x='Physical_Activity_Hours_Per_Day', y='GPA', data=df, ax=axes[1], alpha=0.6, color='green')
axes[1].set_title('Physical Activity Hours vs. GPA', fontsize=14)
axes[1].set_xlabel('Physical Activity Hours Per Day', fontsize=12)
axes[1].set_ylabel('GPA', fontsize=12)

plt.tight_layout()
plt.savefig('key_scatter_plots.png')
plt.show()
```





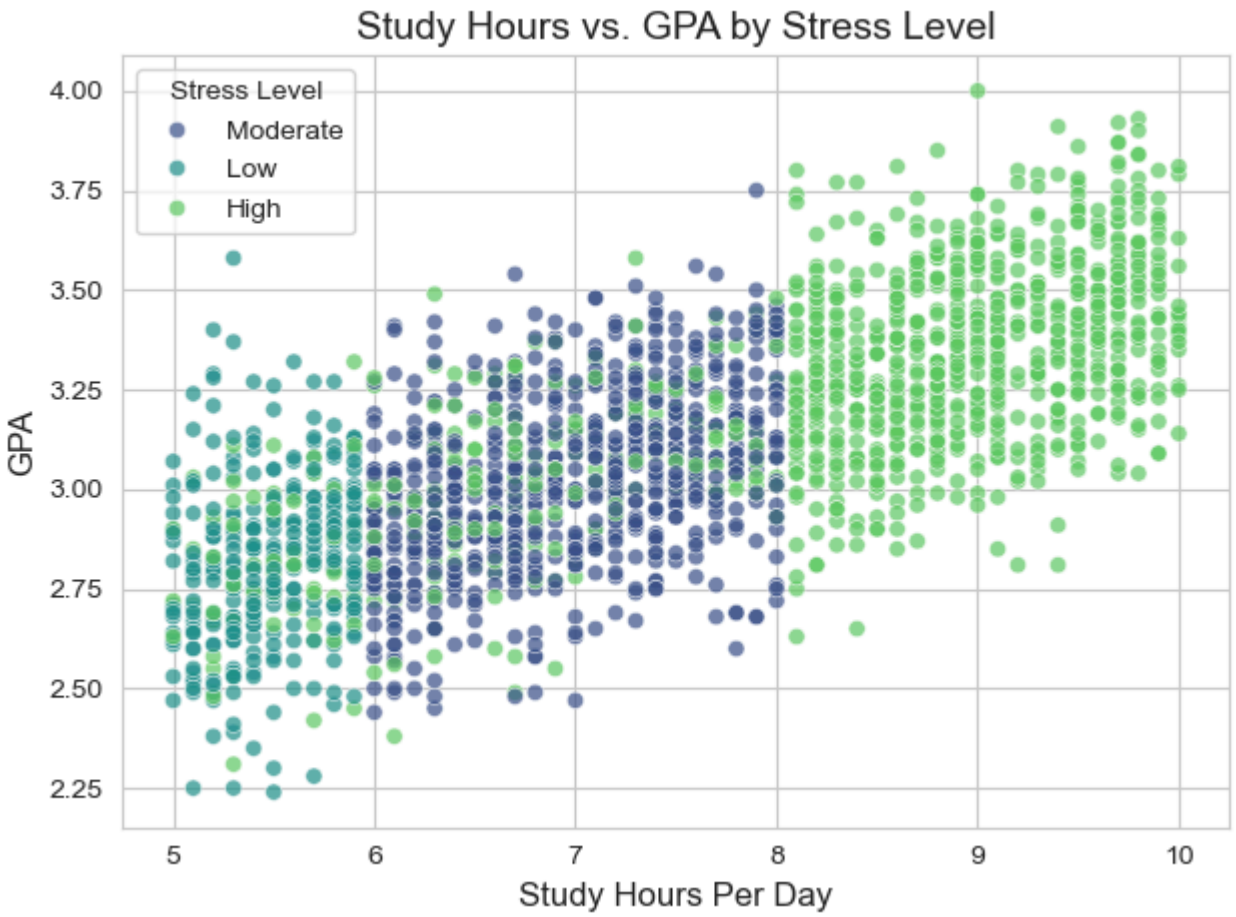
Let's make note of the observations from all the scatter plots:

- Firstly, for 3 variables i.e. Social_Hours, Extracurricular_Hours and Sleep_hours the spread of data is nearly symmetrical and no visible trends can be observed.
- The variables Physical_Activity_Hours and Study_Hours however do possess observable trends and require more analysis.
- The Study_hours variabls has a clear and strong positive linear relationship with GPA describing that students who spend more hours studying generally achieved higher GPA scores.
- The Physical_Activity_Hours has weak negative liner trned which describes thats the GPA score may slightly decrease with more physical activity. A possible reason for this is trading time off one activity for another.

We will make more use of the study hours variable as it has the strongest visual indications of variable relationships. Stress levels may also have a link in this case so we should explore the possibilty of a multivariate relationship. To investigate this, a multivariate scatter plot can be created.

In [13]:
```
#This will recreate the study hours vs GPA scatter plot, but this time visually seperate the data by each stress
#level.
sns.scatterplot(x='Study_Hours_Per_Day', y='GPA', hue='Stress_Level', data=df, alpha=0.7, palette='viridis')
plt.title('Study Hours vs. GPA by Stress Level', fontsize=14)
plt.xlabel('Study Hours Per Day', fontsize=12)
plt.ylabel('GPA', fontsize=12)
plt.legend(title='Stress Level')

plt.tight_layout()
```

```
plt.savefig('key_scatter_plots.png')
plt.show()
```



In this variation of the Study_Hours vs GPA scatter plot, we see:

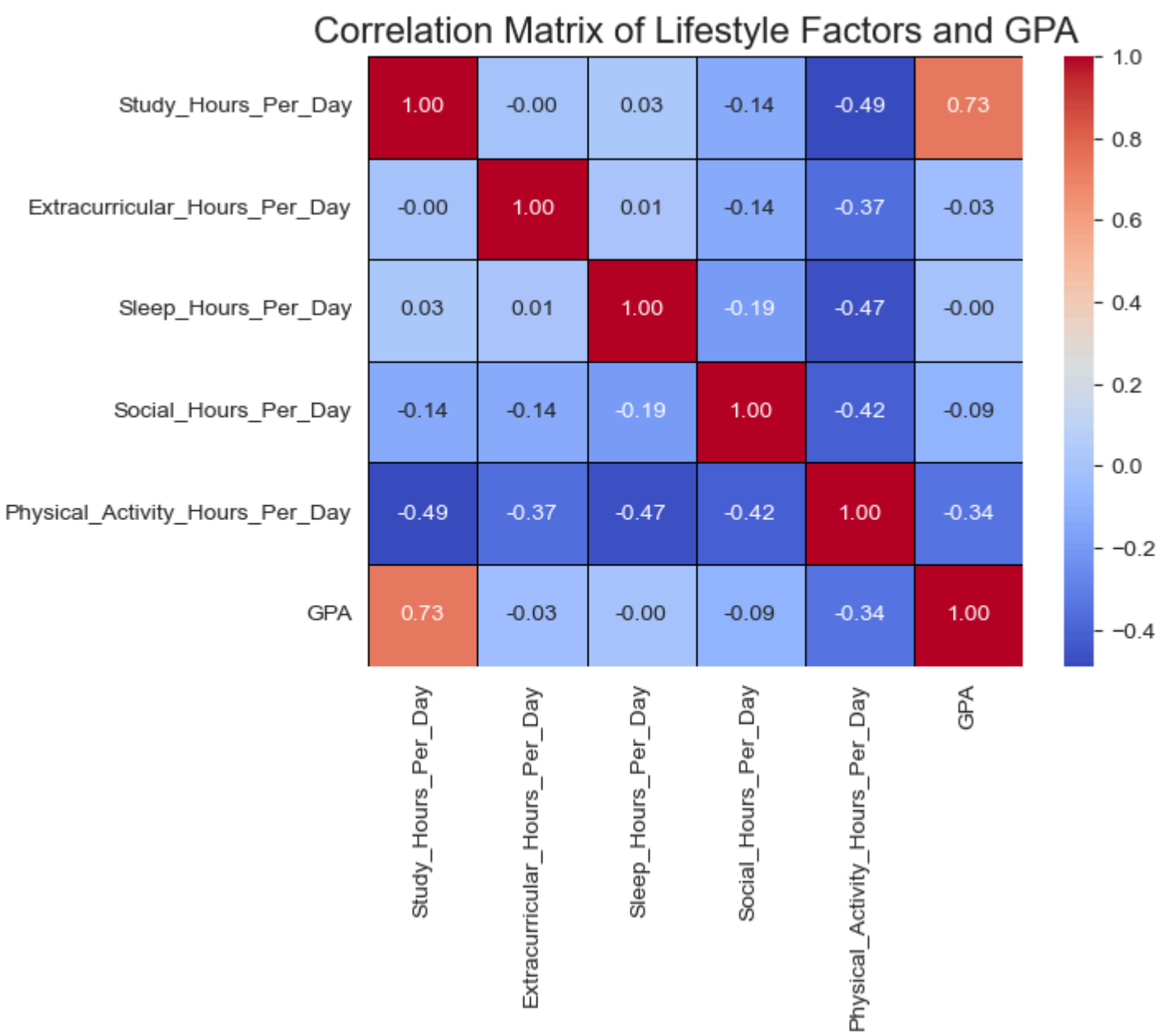
- Lower stress levels tend to cluster near the area of lower study hours and lower GPA.
- Higher stress levels tend to cluster towards the area of higher study hours and higher GPA.
- This tells us that the relatinship between Study hours and GPA exists even within individual stress groups and that the high stress group contains the hardest working and highest achieving students.

Perhaps we should try other visual representations to highlight relationships between variables for extra practice and to help reinforce our claims about what the dataset tells us.

Lets try another strong indicator of variable relationships, the correlation matrix heatmap.

In [14]:

```
#Initial step is to calculate the correlation matrix
corr_matrix =  df[numeric_cols].corr()

#Now to set up the heatmap, we want to show the correlation value so we will set 'annot=True' and we will use a
#color scheme of 'coolwarm'
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt=".2f", linewidths=.5, linecolor='black')
plt.title('Correlation Matrix of Lifestyle Factors and GPA', fontsize=16)
plt.savefig('correlation_heatmap.png')
plt.show()
```



A heatmap matrix shows all linear relationships scaled from -1.0 (perfect negative correlation to +1.0 (perfect positive correlation), there we can deduct from the figure that:

- There is a strong positve correlation between Study hours and GPA (0.73) reinforcing our previous claims.
- There is a negative correlation between physical activty hours and GPA (-0.34) which also reinforces our previous claims.
- Other variables have too weak correlations with regards to GPA to consider regardless if it is postive or negative.

- There is a special relationship to be noted, the study hours and physical hours variables feature a strong negative correlation (-0.49), this substantiates that time trade off event we mentioned earlier, meaning less time on physical activity is more time to study and vice versa.

Now that we have explored various ways to explore our data and are satisfied with correlations and assumptions declared we can move on to the next step in the process.

STEP 4 : Building statistical models.

We begin by importing additional libraries and modules that are required for data manipluation and machine learning.

In [15]:
```
#The 'train_test_split function' is required to divide data into training and testing sets, 'linearRegression' is
#the specific mdoel that will be used to predict the GPA variable and 'mean_absolute_error and 'r2_score' are
#metrics that depict how well the regression model performs.
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, r2_score
```

Next is data preprocessing.

In [16]:
```
#The student_ID column is removed and Stress_level is converted to a numerical column. The category high is dropped
#as it can be represented by when Low and Moderate are 0.
df_processed = df.drop('Student_ID', axis=1)
df_processed = pd.get_dummies(df_processed, columns=['Stress_Level'], drop_first=True)
```

Defining the Features(x) and Target(y).

In [17]:
```
#x will represent the independent variables and y will represent the dependant variable GPA.
x = df_processed.drop('GPA', axis=1)
y = df_processed['GPA']
```

Splitting the data.

- 80% of the data will be used to train the model.
- The remaing 20% of the data will be reserved for testing.

In [18]:
```
#The train_test_split function will randomly split the data into 4 susbsets, test_size specifies the amount data to
#be allocated to testing and random_state ensure the split is the same every time the code is ran.
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=42)
```

Start training the model.

In [19]:
```
#This is the core learning step in which the model uses the training dat to calculate the weight and slope
#coefficients that will best fit the linear relationship between x and y.
model = LinearRegression()
model.fit(x_train, y_train)
```

Out[19]:

LinearRegression

▶ Parameters

Now we can make predictions and evaluate them.

In [20]:
```
#The model will use the learned coefficients to generate predicitions using the test values of x, the mean absolute
#error measures the difference between the predicted y value and the test y value (lower is better) and the
#R-squared score tells us the percentage of variation in the GPA that the model is capable of explaining( value
#close to 1.0 is best).
y_pred = model.predict(x_test)
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
print("R-squared score = " + str(round(r2, 4)) + " and the mean absolute error = " + str(round(mae, 4)))
```

```
R-squared score = 0.5474 and the mean absolute error = 0.1644
```

And lastly we analyze the coefficients created.

In [21]:
```
#The dataframe table created will organize the model's coefficients. The magnitude of positivity or negativity
#reveal the effect an x variable has on the y GPA.
coefficients_df = pd.DataFrame(model.coef_, x_train.columns, columns=['Coefficient'])
print(coefficients_df.sort_values(by='Coefficient', ascending=False).to_markdown(numalign="left", stralign="left"))
```

	Coefficient
Study_Hours_Per_Day	0.124126
Stress_Level_Low	0.00460563
Stress_Level_Moderate	-0.0232416
Social_Hours_Per_Day	-0.0275197
Physical_Activity_Hours_Per_Day	-0.0282109
Sleep_Hours_Per_Day	-0.0300979
Extracurricular_Hours_Per_Day	-0.0382976

To conclude the performance of the model, a predictive capability 54.74% (R-squared) is decent enough to draw insight but it depicts that their are probably external factors that also play a role in GPA achievements (intelligence, teaching quality, school preparation), the MAE tells us that a predict value will be 0.1644 GPA points off approximately (eg. if actual is 3.0 then predicted may be between 2.84 and 3.16). The coefficients for x variables serve to reinforce the ideas we had about the dataset initially during exploration, as they depict the same relationships.

We will try building another model, this time the model will make predictive classifications of stress levels using Multinomial Logistic Regression. Let's start by import the required libraries and modules.

In [22]:
```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report
```

Now we define the feature x and the target z.

In [23]:
```
x_class = df.drop(['Student_ID', 'Stress_Level'], axis=1)

z = df['Stress_Level']
```

Then the data is split using stratification function into the same proportions of Low, Moderate and High.

In [24]:
```
#stratify=z ensures the split has the same proportionality.
x_train_class, x_test_class, z_train, z_test = train_test_split(
    x_class, z, test_size=0.2, random_state=42, stratify=z
)
```

Let the model training begin!

In [25]:
```
# iterations is set to 1000.
logreg_model = LogisticRegression(max_iter=1000, random_state=42)
logreg_model.fit(x_train_class, z_train)
```

Out[25]:

▾ LogisticRegression

▶ Parameters

To finalize, we run the predictive model and evaluate it.

In [26]:
```
z_pred = logreg_model.predict(x_test_class)
accuracy = accuracy_score(z_test, z_pred)


print(f"\n--- Model Evaluation (Stress Level Prediction) ---")
print(f"Overall Classification Accuracy: {accuracy:.4f}\n")
print("Detailed Classification Report (Precision, Recall, F1-Score):")
print(classification_report(z_test, z_pred))
```

```
--- Model Evaluation (Stress Level Prediction) ---
Overall Classification Accuracy: 0.8475

Detailed Classification Report (Precision, Recall, F1-Score):
              precision    recall  f1-score   support

        High       0.86      0.88      0.87       206
         Low       0.84      0.92      0.88        59
    Moderate       0.83      0.77      0.80       135

    accuracy                           0.85       400
   macro avg       0.84      0.85      0.85       400
weighted avg       0.85      0.85      0.85       400
```

The results tell us that the overall accuracy is 0.8475 which means that the model is able to predict a student's stress level accurately roughly 85 out of 100 times on an unseen dataset, this is therefore a highly accurate model. The detailed performance highlights which classification the model is better at identifying, the higher the value the better.

We can make two final conclusions about the dataset that indicate that lifestyle choices are strong indicators of a student's GPA outcomes

- GPA is primarily driven by Study hours, and can be negatively impacted by trading time off doing other activities.
- The stress level of a student is highly predictable based on their lifestyle factors and GPA.